

Comprehensive Summary

Transformers Fundamentals

Course: AI103

Overview

=====

Transformers are a type of neural network architecture that have revolutionized the field of natural language processing (NLP) and beyond. They rely entirely on self-attention mechanisms to draw global dependencies between input and output sequences, making them highly effective in various NLP tasks. Understanding the fundamentals of transformers is crucial for students to appreciate their applications and limitations in modern AI systems.

Definitions

=====

- **Transformers**: A type of neural network architecture that relies entirely on self-attention mechanisms to draw global dependencies between input and output sequences.
- **Self-Attention Mechanism**: A mechanism that allows the model to attend to different parts of the input sequence simultaneously and weigh their importance.
- **Encoder-Decoder Structure**: A structure that consists of an encoder and a decoder, where the encoder maps an input sequence to a sequence of continuous representations, and the decoder generates an output sequence of symbols one element at a time.
- **Multi-Head Attention**: A mechanism that involves linearly projecting the queries, keys, and values multiple times with different, learned linear projections to compute the attention function in parallel.
- **Scaled Dot-Product Attention Mechanism**: A particular attention mechanism used in the transformer model, which computes the attention function as a weighted sum of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.

Main Topics

=====

1. ****Introduction to Transformers****

The transformer model is a type of neural network architecture that relies entirely on self-attention mechanisms to draw global dependencies between input and output sequences. The model consists of an encoder and a decoder, where the encoder maps an input sequence of symbol representations to a sequence of continuous representations, and the decoder generates an output sequence of symbols one element at a time.

Key Points:

- The transformer model uses stacked self-attention and point-wise, fully connected layers for both the encoder and decoder.
- The output of each sub-layer is given by $\text{LayerNorm}(x + \text{Sublayer}(x))$.
- The transformer model has been shown to be highly effective in various NLP tasks, including machine translation, text classification, and language modeling.

Examples:

→ A company wants to develop a chatbot that can understand and respond to customer queries. The company can use a transformer model to analyze the customer queries and generate responses.

→ A researcher wants to analyze a large dataset of text documents to identify patterns and trends. The researcher can use a transformer model to extract features from the text documents and perform clustering or classification tasks.

2. ****Key Components of Transformers****

The key components of a transformer architecture include tokenization, word embedding, and multi-head attention. Tokenization involves converting text into numerical representations called tokens, while word embedding involves converting each token into a vector via lookup from a word embedding table.

Key Points:

- Tokenization is the process of converting text into numerical representations called tokens.
- Word embedding is the process of converting each token into a vector via lookup from a word embedding table.
- Multi-head attention involves contextualizing each token within the scope of the context window with other (unmasked) tokens via a parallel multi-head attention mechanism.

Examples:

→ A text classification task involves classifying a piece of text into one of several categories. A transformer model can be used to extract features from the text and perform classification.

3. ****Attention Mechanism****

The attention function is a mapping of a query and a set of key-value pairs to an output. The output is computed as a weighted sum of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.

Key Points:

- The attention function is computed as a weighted sum of the values.
- The weight assigned to each value is computed by a compatibility function of the query with the corresponding key.
- The attention mechanism allows the model to attend to different parts of the input sequence simultaneously and weigh their importance.

Examples:

→ A machine translation task involves translating a piece of text from one language to another. A transformer model can be used to attend to different parts of the input sequence and generate the translated text.

4. **Multi-Head Attention**

Instead of performing a single attention function, the transformer model uses multi-head attention. This involves linearly projecting the queries, keys, and values multiple times with different, learned linear projections to compute the attention function in parallel.

Key Points:

- Multi-head attention involves linearly projecting the queries, keys, and values multiple times with different, learned linear projections.
- The attention function is computed in parallel on each of these projected versions, yielding multiple output values.
- The output values are concatenated and once again projected, resulting in the final output.

Examples:

→ A language modeling task involves predicting the next word in a sequence of text. A transformer model can be used to attend to different parts of the input sequence and predict the next word.

5. **Applications of Transformers**

Transformers have been widely adopted for various applications, including language modeling, text classification, machine translation, and computer vision.

Key Points:

- Transformers have been shown to be highly effective in various NLP tasks, including machine translation, text classification, and language modeling.

- Transformers can be used for other tasks, such as computer vision and audio processing.
- The transformer model's ability to handle parallelization and reduce training time has made it an attractive choice for large-scale NLP tasks.

Examples:

→ A company wants to develop a language model that can generate coherent and natural-sounding text. The company can use a transformer model to train the language model.

Equations

=====

1. **Attention Function:** The attention function is computed as a weighted sum of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) * V$$

where Q, K, and V are matrices representing the queries, keys, and values, respectively.

2. **Multi-Head Attention:** The multi-head attention is computed as:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) * W^O$$

$$\text{where } \text{head}_i = \text{Attention}(QW^{Q_i}, KW^{K_i}, VW^{V_i})$$

and the projections are parameter matrices $W^{Q_i} \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W^{K_i} \in \mathbb{R}^{d_{\text{model}} \times d_k}$, and $W^{V_i} \in \mathbb{R}^{d_{\text{model}} \times d_v}$.

3. **Scaled Dot-Product Attention Mechanism:** The scaled dot-product attention mechanism is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) * V$$

where Q, K, and V are matrices representing the queries, keys, and values, respectively.

Example: Given a query vector Q, a key vector K, and a value vector V, the attention function can be computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) * V$$

where Q, K, and V are matrices representing the queries, keys, and values, respectively.